

XBoing Modernized Architecture

SDL2-Based Codebase Architecture Document
For AI-Assisted Legacy Modernization Research

XBoing Modernization Project

September 2026 — Revision 1.1

Table 1: Revision History

Rev	Date	Description
1.0	2026-03-24	Initial architecture document covering completed modernization
1.1	2026-09-27	Fact-check against release 1.0.11: ADR summary extended to ADR-074; CI workflow count corrected to five; cross-reference to the modernization case study added

Contents

1	Introduction	3
1.1	Modernization Principles	3
1.2	Structural Overview	3
2	Build System	3
2.1	Dependency Discovery	3
2.2	Library Targets	4
2.3	Compiler Warning Policy	4
2.4	CMake Presets	4
2.5	FHS Install Layout	4
3	Architecture Overview	4
4	SDL2 Platform Layer	5
4.1	Renderer (<code>sd12_renderer</code>)	5
4.2	Texture Cache (<code>sd12_texture</code>)	5
4.3	Font System (<code>sd12_font</code>)	6
4.4	Color System (<code>sd12_color</code>)	6
4.5	Cursor Management (<code>sd12_cursor</code>)	6
4.6	Logical Regions (<code>sd12_regions</code>)	7
4.7	Audio (<code>sd12_audio</code>)	7
4.8	Input Mapping (<code>sd12_input</code>)	7
4.9	State Machine Engine (<code>sd12_state</code>)	7
4.10	Game Loop (<code>sd12_loop</code>)	8
4.11	CLI Parser (<code>sd12_cli</code>)	8
5	Pure-C System Libraries	8

5.1	Ball System (<code>ball_system</code>)	9
5.2	Block System (<code>block_system</code>)	10
5.3	Paddle System (<code>paddle_system</code>)	10
5.4	Gun System (<code>gun_system</code>)	10
5.5	Score System (<code>score_system</code>)	10
5.6	Level System (<code>level_system</code>)	11
5.7	Special System (<code>special_system</code>)	11
5.8	Bonus System (<code>bonus_system</code>)	11
5.9	SFX System (<code>sfx_system</code>)	11
5.10	EyeDude System (<code>eyedude_system</code>)	11
5.11	Message System (<code>message_system</code>)	11
5.12	Editor System (<code>editor_system</code>)	11
5.13	UI Sequencers	11
6	Integration Layer	12
6.1	Game Context (<code>game_ctx_t</code>)	12
6.2	Initialization (<code>game_init.c</code>)	12
6.3	Callback Wiring (<code>game_callbacks.c</code>)	13
6.4	Mode Handlers (<code>game_modes.c</code>)	14
6.5	Rendering (<code>game_render.c</code> + <code>game_render_ui.c</code>)	14
6.6	Input Dispatch (<code>game_input.c</code>)	14
6.7	Game Rules (<code>game_rules.c</code>)	14
7	Persistence	14
7.1	High Score I/O (<code>highscore_io</code>)	14
7.2	Save Game I/O (<code>savegame_io</code>)	14
7.3	Config I/O (<code>config_io</code>)	15
7.4	Path Resolution (<code>paths</code>)	15
8	Design Patterns	15
8.1	Opaque Context Pattern	15
8.2	Callback Injection	15
8.3	Caller-Provided Buffers	16
8.4	Best-Effort Loading	16
8.5	Enum-Indexed Lookups	16
8.6	Environment Struct Pattern	16
9	Testing Infrastructure	16
9.1	Test Framework	16
9.2	Headless Testing	16
9.3	Sanitizer Builds	17
9.4	Fuzz Testing	17
9.5	Integration Test Categories	17
10	CI/CD	17
11	Module Dependency Map	17
12	Legacy Comparison	17
13	Architectural Decision Records Summary	18
14	Glossary	20

1 Introduction

XBoing is a breakout/blockout game originally written by Justin C. Kibell in 1993–1996 for X11 using raw Xlib, XPM pixmaps, and a fork/pipe audio architecture. The modernization replaces all platform dependencies with SDL2 while preserving the original gameplay, physics, scoring, and level format. The result is a Linux game that builds with standard SDL2 packages, follows FHS and XDG conventions, and plays identically to the original.

This document describes the *architecture* of the modern codebase. For the project’s *method* — how the modernization was carried out with LLM agents, ethos mission contracts, and z-spec formal methods — see the companion case study `docs/MODERNIZATION_CASE_STUDY.tex`.

1.1 Modernization Principles

Five principles governed every decision:

1. **Test before you change, not after.** Characterization tests capture existing behavior before any modification.
2. **Never rewrite from scratch.** Replace one subsystem at a time; prove equivalence with tests.
3. **Separate concerns in commits.** A modernization commit must not also fix a bug or add a feature.
4. **Preserve original behavior first.** Game constants (`MAX_BALLS=5`, grid 18×9 , paddle sizes `40/50/70`, `DIST_BASE=30`, all scoring values) are preserved exactly.
5. **Document architectural decisions.** All non-trivial choices are recorded as ADRs in `docs/DESIGN.md` (section 13).

1.2 Structural Overview

The codebase is organized into three directories: `src/*.c` (source), `include/*.h` (headers), and `tests/` (unit tests, integration tests, fuzz harnesses). Platform and game system modules each compile to a static library; the integration layer (`game_*.c`) compiles directly into the executable, which links all libraries. Three CMake presets (`debug`, `asan`, `install`) cover development, sanitizer, and installation builds. ADRs are recorded in `docs/DESIGN.md`.

2 Build System

CMake is the sole build system (ADR-001). The root `CMakeLists.txt` defines static library targets and 1 executable; test targets are delegated to `tests/CMakeLists.txt`.

2.1 Dependency Discovery

SDL2 libraries are discovered via `pkg_check_modules`:

Table 2: SDL2 dependencies

Package	CMake Variable	Purpose
<code>SDL2</code>	<code>SDL2</code>	Window, renderer, input, timer
<code>SDL2_image</code>	<code>SDL2_IMAGE</code>	PNG texture loading
<code>SDL2_mixer</code>	<code>SDL2_MIXER</code>	WAV audio playback
<code>SDL2_ttf</code>	<code>SDL2_TTF</code>	TTF font rendering
<code>cmocka</code>	<code>CMOCKA</code>	Unit test framework

2.2 Library Targets

Each module compiles to a static library (.a). Libraries fall into five categories:

SDL2 Platform sdl2_renderer, sdl2_texture, sdl2_font, sdl2_color, sdl2_cursor, sdl2_regions, sdl2_audio, sdl2_input

Pure C Engine sdl2_state, sdl2_loop, sdl2_cli, score_logic

Game Systems ball_system, block_system, paddle_system, gun_system, score_system, level_system, special_system, bonus_system, sfx_system, eyedude_system, message_system, editor_system

UI Sequencers presents_system, intro_system, demo_system, keys_system, dialogue_system, highscore_system

Persistence highscore_io, savegame_io, config_io, paths

One executable links all libraries: xboing, built only when all four SDL2 packages are found.

2.3 Compiler Warning Policy

All targets compile with -Werror and the following warning set:

```
-Wall -Wextra -Wpedantic -Wconversion -Wshadow  
-Wdouble-promotion -Wformat=2 -Wformat-overflow=2 (GCC only)  
-Wnull-dereference -Wuninitialized  
-Wstrict-prototypes -Wold-style-definition
```

2.4 CMake Presets

Table 3: CMake presets (CMakePresets.json)

Preset	Build Dir	Configuration
debug	build/	CMAKE_BUILD_TYPE=Debug
asan	build-asan/	Debug + -fsanitize=address,undefined -fno-omit-frame-pointer
install	build-install/	Release, CMAKE_INSTALL_PREFIX=/usr, tests disabled

2.5 FHS Install Layout

The install preset (ADR-003) produces an FHS-compliant installation:

```
/usr/bin/xboing  
/usr/share/xboing/levels/*.data  
/usr/share/xboing/sounds/*.wav  
/usr/share/xboing/fonts/*.ttf  
/usr/share/man/man6/xboing.6  
/usr/share/applications/xboing.desktop  
/usr/share/metainfo/com.github.xboing.metainfo.xml  
/usr/share/icons/hicolor/{16,32,48,128,256}x.../xboing.png
```

3 Architecture Overview

The modern codebase is organized in three layers. The SDL2 platform layer manages hardware interaction. Pure-C system libraries implement game logic with zero platform dependency. The integration layer (game_*.c files) wires the two together through the central game_ctx_t struct.

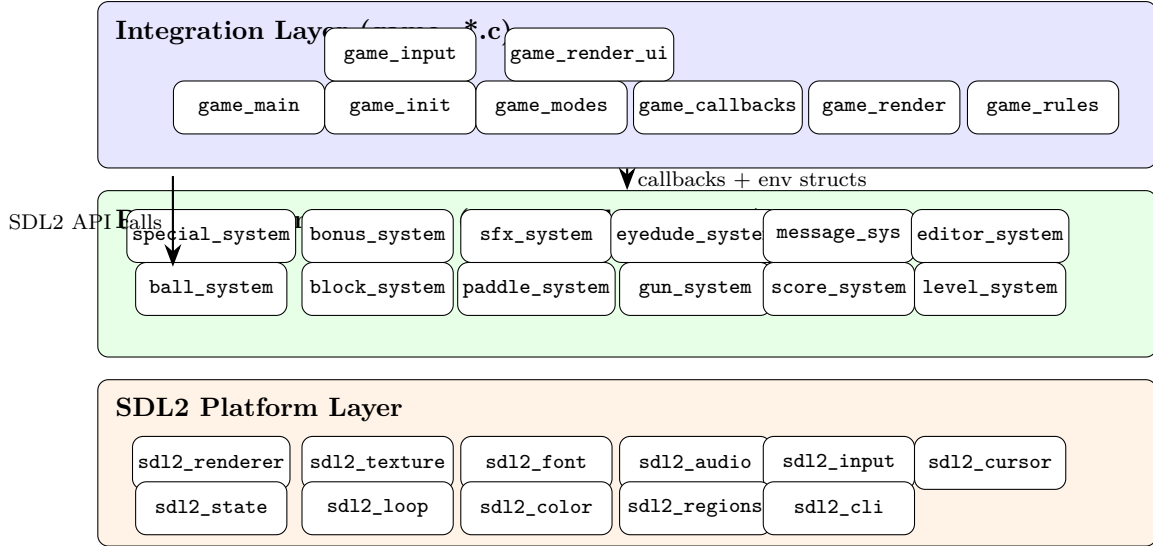


Figure 1: Three-layer architecture. Pure-C systems never call SDL2 functions; they communicate side effects through injected callback tables. The integration layer bridges both.

The defining architectural choice is **callback injection**: pure-C system libraries accept function pointer tables at creation time and invoke them for side effects (sound, score changes, block hits). The integration layer provides callback implementations that route to other systems or to SDL2 platform modules. This eliminates the 50+ `extern` globals that coupled legacy modules.

4 SDL2 Platform Layer

Each SDL2 platform module follows the opaque context pattern: a heap-allocated struct hidden behind an opaque typedef, accessed only through public API functions.

4.1 Renderer (`sdl2_renderer`)

Manages the SDL2 window and GPU-accelerated renderer (ADR-004).

Table 4: Renderer configuration

Parameter	Value
Logical resolution	575×720 pixels
Default scale	2× (physical 1150×1440)
Window flags	SDL_WINDOW_SHOWN SDL_WINDOW_RESIZABLE
Fullscreen mode	SDL_WINDOW_FULLSCREEN_DESKTOP (toggle via I key)
Scaling filter	SDL_HINT_RENDER_SCALE_QUALITY = "nearest"
VSync	Enabled by default (SDL_RENDERER_PRESENTVSYNC)
Fallback	Software renderer if GPU acceleration unavailable

The 575×720 logical resolution matches the legacy `mainWindow` dimensions exactly: `PLAY_WIDTH` (495) + `MAIN_WIDTH` (70) + 10 by `PLAY_HEIGHT` (580) + `MAIN_HEIGHT` (130) + 10. `SDL_RenderSetLogicalSize()` handles letterboxing.

4.2 Texture Cache (`sdl2_texture`)

Loads all PNG assets into an FNV-1a hash map for O(1) lookup by string key (ADR-005).

- **Hash map:** 256 slots, open addressing with linear probing, $\sim 70\%$ load factor for ~ 180 textures.
- **Keys:** Relative path with `.png` stripped (e.g., `"balls/ball1"`).
- **Loading:** Eager—all PNGs loaded at init via recursive `opendir()/readdir()` directory walk.
- **Partial load tolerance:** Individual file failures logged but do not abort cache creation.
- **FNV-1a parameters:** Offset basis 2166136261, prime 16777619.

The hash map is write-once at initialization and read-many per frame.

4.3 Font System (`sd12_font`)

Replaces four XLFD bitmap fonts with Liberation Sans TTF via `SDL2_TTF` (ADR-006).

Table 5: Font slots matching legacy fonts

Enum	Legacy XLFD	Size	Style
<code>SDL2F_TITLE</code>	<code>adobe-helvetica-bold-24</code>	24pt	Bold
<code>SDL2F_TEXT</code>	<code>adobe-helvetica-medium-18</code>	18pt	Regular
<code>SDL2F_DATA</code>	<code>adobe-helvetica-bold-14</code>	14pt	Bold
<code>SDL2F_COPY</code>	<code>adobe-helvetica-medium-12</code>	12pt	Regular

Text rendering uses the transient texture pattern: `TTF_RenderUTF8_Blended()` $\rightarrow$ `SDL_CreateTextureFromSurface()` $\rightarrow$ `SDL_RenderCopy()` $\rightarrow$ destroy. Shadow text renders at $(x+2, y+2)$ in black, then (x, y) in the foreground color. At ~ 20 text draws per frame, the overhead is negligible.

4.4 Color System (`sd12_color`)

Provides precomputed RGBA values verified against X11 `rgb.txt` (ADR-007). Unlike other `SDL2` modules, this has **no opaque context**—all data is `static const`.

- 8 named colors: RED, TAN, YELLOW, GREEN, WHITE, BLACK, BLUE, PURPLE.
- 2 gradient arrays: `sd12_color_red_gradient(i)` and `sd12_color_green_gradient(i)`, each 7 steps, matching legacy hex values (`#f00` through `#300`).
- String lookup via `sd12_color_by_name()` for migration from `ColourNameToPixel()` call sites.

4.5 Cursor Management (`sd12_cursor`)

Pre-creates 5 cursor types at init (ADR-008), switching via $O(1)$ `SDL_SetCursor()`:

Table 6: Cursor mapping from legacy to `SDL2`

Enum	Legacy X11	<code>SDL2</code> System Cursor
<code>SDL2CUR_WAIT</code>	<code>XC_watch</code>	<code>SDL_SYSTEM_CURSOR_WAIT</code>
<code>SDL2CUR_PLUS</code>	<code>XC_plus</code>	<code>SDL_SYSTEM_CURSOR_CROSSHAIR</code>
<code>SDL2CUR_POINT</code>	<code>XC_hand2</code>	<code>SDL_SYSTEM_CURSOR_HAND</code>
<code>SDL2CUR_SKULL</code>	<code>XC_pirate</code>	<code>SDL_SYSTEM_CURSOR_CROSSHAIR</code>
<code>SDL2CUR_NONE</code>	blank pixmap	<code>SDL_ShowCursor(SDL_DISABLE)</code>

Best-effort creation: missing cursors (e.g., dummy driver) are stored as `NULL`; `set()` gracefully skips `SDL_SetCursor` while still tracking the active cursor ID.

4.6 Logical Regions (`sd12_regions`)

Replaces 10 X11 sub-windows with 9 named `SDL_Rect` constants in the 575×720 logical coordinate system (ADR-009). No opaque context—pure static data.

Table 7: Logical render regions (all coordinates in pixels)

Region	X	Y	W	H
PLAY	35	60	495	580
SCORE	35	10	224	42
LEVEL	284	5	286	52
MESSAGE	35	655	247	30
SPECIAL	292	655	180	35
TIMER	477	655	61	35
EDITOR	Editor-specific block palette			
EDITOR_TYPE	Editor-specific type display			
DIALOGUE	Centered modal input overlay			

Hit testing via `sd12_region_hit_test(x, y)` maps pixel coordinates to region IDs for mouse dispatch.

4.7 Audio (`sd12_audio`)

Replaces the fork/pipe architecture and 12 platform-specific drivers with `SDL2_mixer` (ADR-010).

- **Eager WAV caching:** Scans sound directory at init, loads all `.wav` files into an FNV-1a hash map keyed by basename (e.g., "boing").
- **Concurrent playback:** `Mix_PlayChannel(-1, chunk, 0)` auto-selects from 16 allocated channels. Legacy could only play one sound at a time.
- **Global volume:** `Mix_Volume(-1, vol)` applies to all channels.
- **CI testing:** `SDL_AUDIODRIVER=dummy` enables headless operation.

4.8 Input Mapping (`sd12_input`)

Replaces `XLookupString()` `KeySym` dispatch with a scancode-to-action mapping layer (ADR-011).

- Game actions covering movement, gameplay, menu, global controls, and 9 speed levels (see `sd12_input_action_t`).
- Dual binding slots per action (e.g., Left arrow + J for paddle left).
- O(1) reverse lookup table: `SDL_Scancode` → `action` array (`SDL_NUM_SCANCODES` entries).
- Level trigger (`pressed()`) for held keys; edge trigger (`just_pressed()`) for one-shot actions.
- Mouse position and button state from `SDL_MOUSEMOTION`/`SDL_MOUSEBUTTONDOWN`.
- Runtime rebindable via `bind()/reset_bindings()`.

4.9 State Machine Engine (`sd12_state`)

Replaces the legacy `switch(mode)` dispatch with a function pointer table (ADR-012). Pure C—no `SDL2` dependency.

Each mode registers three optional callbacks via `sd12_state_mode_def_t`: `on_enter`, `on_update`, `on_exit`. Transitions call `exit` on the old mode, then `enter` on the new. Dialogue

Table 8: Game modes (16 values matching legacy `MODE_*` defines)

Value	Mode
0	NONE
1	HIGHSCORE
2	INTRO
3	GAME
4	PAUSE
5	BALL_WAIT (legacy: never assigned)
6	WAIT (legacy: never assigned)
7	BONUS
8	INSTRUCT
9	KEYS
10	PRESENTS
11	DEMO
12	PREVIEW
13	DIALOGUE
14	EDIT
15	KEYSEEDIT

mode uses push/pop semantics (`push_dialogue()`/`pop_dialogue()`) to save and restore the interrupted mode.

The frame counter increments on every `update()` except in PAUSE and DIALOGUE modes.

4.10 Game Loop (`sd12_loop`)

Fixed-timestep accumulator replacing legacy `usleep()` timing (ADR-013). Pure C—no SDL2 dependency.

Table 9: Speed level tick rates

Speed	Tick Interval	Ticks/sec
1 (slowest)	13,500 μ s	$\sim$ 74
5 (default)	7,500 μ s	$\sim$ 133
9 (fastest)	1,500 μ s	$\sim$ 667

4.11 CLI Parser (`sd12_cli`)

Pure C command-line parser (ADR-014). Populates a `sd12_cli_config_t` struct with no side effects. Legacy X11-specific options (`-display`, `-geometry`, etc.) are dropped. Status codes distinguish informational exits (`EXIT_HELP`, `EXIT_VERSION`) from errors (`ERR_MISSING_VALUE`, `ERR_INVALID_VALUE`).

5 Pure-C System Libraries

All game system modules share these properties:

1. **Opaque context:** Heap-allocated via `calloc()`, destroyed via `free()`. No global or static mutable state.

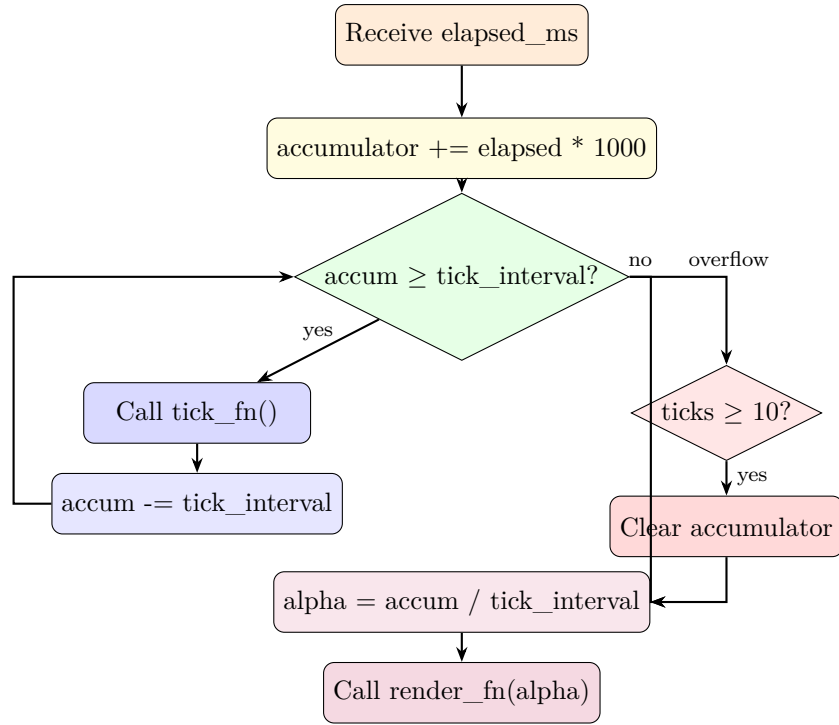


Figure 2: Fixed-timestep game loop. Tick interval = $1500 \times (10 - \text{speed_level})$ microseconds. Spiral-of-death prevention caps at 10 ticks per frame.

2. **Callback injection:** Side effects communicated through function pointer tables provided at creation. Any callback may be NULL (no-op).
3. **Environment structs:** Per-frame state from other systems passed as const structs, never cached.
4. **No SDL2 or X11 dependency:** Links only `libc` (and `libm` for ball physics).
5. **Render info queries:** Position, animation frame, and visibility data returned through caller-provided structs. The system never renders.

5.1 Ball System (`ball_system`)

Owns the BALL array (up to `MAX_BALLS=5`), state machine dispatch, physics, and queries (ADR-015).

State machine per ball: `BALL_CREATE` → `BALL_READY` (sitting on paddle) → `BALL_ACTIVE` (in play) → `BALL_POP` (death animation) → `BALL_NONE` (slot freed).

Environment struct (`ball_system_env_t`) carries 12 values replacing legacy extern globals:

```

typedef struct {
    int frame, speed_level, paddle_pos, paddle_dx, paddle_size;
    int play_width, play_height; /* 495, 580 */
    int no_walls, killer, sticky_bat;
    int col_width, row_height; /* 55, 32 */
} ball_system_env_t;

```

Callback table (7 function pointers):

```

check_region Block collision check (replaces XRectInRegion())
on_block_hit Block hit dispatch (replaces HandleTheBlocks())
cell_available Grid query for hyperspace teleport

```

on_sound Sound effect playback
on_score Point award
on_message Status bar update
on_event Lifecycle events (died, split, tilt, paddle hit)

Physics functions are extracted to **ball_math.c**: paddle bounce (trigonometric angle calculation), speed normalization, ball-to-ball elastic collision.

5.2 Block System (**block_system**)

Owns the 18×9 block grid with pure-C diagonal cross-product collision replacing X11 Regions (ADR-016).

Collision algorithm: The block rectangle is divided into 4 triangles by its diagonals. Two cross-product values determine the hit face:

$$d_1 = w \cdot (b_y - y) - h \cdot (b_x - x) \quad (\text{TL-BR diagonal}) \quad (1)$$

$$d_2 = h \cdot (x + w - b_x) - w \cdot (b_y - y) \quad (\text{TR-BL diagonal}) \quad (2)$$

Table 10: Cross-product region classification

d_1	d_2	Hit Face
≤ 0	≥ 0	TOP
≥ 0	≤ 0	BOTTOM
≥ 0	≥ 0	LEFT
≤ 0	≤ 0	RIGHT

Three-step collision check: (1) AABB overlap test, (2) diagonal cross-product for hit face, (3) adjacency filter suppresses bounce if neighbor in that direction is occupied.

The block info catalog stores metadata for all 30 block types (point values, animation frames, explode behavior).

5.3 Paddle System (**paddle_system**)

Owns paddle position, size (3 sizes: 40/50/70 pixels), and control flags (ADR-017). Keyboard mode applies **PADDLE_VELOCITY=4** per frame. Mouse mode uses the legacy formula: **pos** = **mouse_x** - (**main_width** / 2) + **half_width**. Reverse controls swap keyboard direction and mirror mouse X.

5.4 Gun System (**gun_system**)

Owns bullet (40 slots) and tink (40 slots) arrays, ammo state, movement physics, and collision dispatch (ADR-018). Normal mode uses slot 0 only; fast gun mode searches all 40 slots, spawning 2 bullets per shot. Collision priority: ball > eyedude > block.

5.5 Score System (**score_system**)

Owns score value, multiplier application, extra life tracking (every 100,000 points), and digit layout computation (ADR-019). Delegates arithmetic to **score_logic**: **score_apply_multiplier()**, **score_extra_life_threshold()**, **score_block_hit_points()**. The x2-wins-over-x4 legacy bug is preserved.

Digit layout: right-aligned, rightmost digit at $x=192$ ($224-32$), stride 32px, max 7 digits (9,999,999).

5.6 Level System (`level_system`)

Parses level data files (title, time bonus, 15×9 character grid) and fires `on_add_block(row, col, type, counter_slide)` for each cell (ADR-020). 30 distinct characters map to block types. 80 levels cycle via `level % 80` (mapping 0 to 80). Background cycling: $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 2$.

5.7 Special System (`special_system`)

Owens 8 power-up flags (ADR-021): reverse (display only, owned by paddle system), sticky, saving, fastGun, noWalls, killer, x2, x4. Setting x2 deactivates x4 and vice versa. `turn_off()` clears all except saving (legacy intentional behavior). `get_labels()` returns render info for the 4×2 panel display.

5.8 Bonus System (`bonus_system`)

10-state tally sequence after level completion (ADR-022). Score is pre-committed at sequence entry (animated tally is cosmetic). Save triggers every `SAVE_LEVEL=5` levels relative to starting level. Pure functions `compute_total()` and `should_save()` are independently testable.

5.9 SFX System (`sfx_system`)

5 visual effects (SHAKE, FADE, BLIND, SHATTER, STATIC) plus BorderGlow ambient animation (ADR-023). No rendering—query functions return coordinates and tile data. SHAKE uses `on_move_window` callback. BorderGlow cycles 7-step red/green ramps on 40-frame intervals. STATIC is preserved as the original stub.

5.10 EyeDude System (`eyedude_system`)

Animated character with 6 walk frames per direction, random direction, turn-at-midpoint, AABB collision detection (ADR-024). Awards 10,000 points on ball hit. Constants: `WIDTH=32`, `HEIGHT=32`, `FRAME_RATE=30`, `WALK_SPEED=5`, `TURN_CHANCE=30%`.

5.11 Message System (`message_system`)

Single-line message bar with optional auto-clear delay. Reverts to default message (level title) when delay expires.

5.12 Editor System (`editor_system`)

Grid editing (draw/erase), board transforms (flip/scroll H/V), palette management, save/load via callbacks. Play-test transitions preserve editor state for return.

5.13 UI Sequencers

Six UI sequencer modules implement attract-mode and post-game screens. All follow the sequencer pattern: advance state per frame, expose query functions for render data, fire callbacks for sound and external events.

The attract mode cycle runs: PRESENTS $\rightarrow$ INTRO $\rightarrow$ INSTRUCT $\rightarrow$ DEMO $\rightarrow$ KEYS $\rightarrow$ KEYSEdit $\rightarrow$ PREVIEW $\rightarrow$ HIGHSCORE $\rightarrow$ INTRO (loops).

Table 11: UI sequencer modules

Module	ADR	States / Purpose
presents_system	ADR-025	14 states: flag, credits, letter stamps, sparkle, typewriter text, curtain wipe
intro_system	ADR-026	2 modes (intro + instructions), block descriptions table, sparkle
demo_system	ADR-027	2 modes (demo + preview), ball trail data, random level loading
keys_system	ADR-028	2 modes (game + editor controls), static key binding tables
dialogue_system	ADR-029	4 validation modes (text, numeric, printable, yes/no), input buffer
highscore_system	ADR-030	Title sparkle, row sparkle, specials timing, score display

6 Integration Layer

Eight source files in `src/game_*.c` wire the platform and system layers together. None are compiled as libraries; they are compiled directly into the `xboing` executable.

6.1 Game Context (`game_ctx_t`)

The central struct (`include/game_context.h`) holds pointers to all module contexts plus shared game state. It is passed as `void *user_data` to every callback.

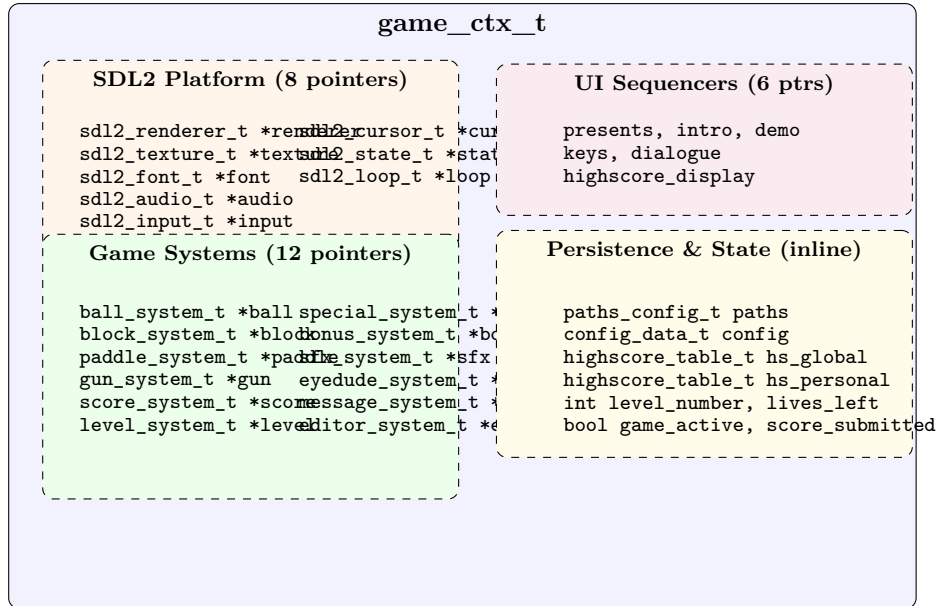


Figure 3: `game_ctx_t` composition. Pointer members reference heap-allocated opaque contexts; persistence and game state fields are owned inline.

Module contexts are forward-declared as opaque typedefs in `game_context.h` to avoid header inclusion cycles. Only `config_io.h`, `highscore_system.h`, and `paths.h` are included (for inline value types).

6.2 Initialization (`game_init.c`)

`game_create(argc, argv)` builds the full context in dependency order:

1. CLI parsing + config file loading + XDG path resolution + high score table loading
2. SDL2 initialization (SDL_Init(VIDEO|AUDIO|TIMER))
3. Platform modules: renderer → texture → font → audio → input → cursor
4. State machine + game loop (with tick/render callbacks)
5. Game systems: block → paddle → ball → gun → score → level → special → bonus → sfx
→ eyedude → message → editor
6. UI sequencers: presents, intro, demo, keys, dialogue, highscore
7. Mode handler registration with the state machine

`game_destroy()` tears down in reverse order.

6.3 Callback Wiring (game_callbacks.c)

The callback injection pattern is the defining architectural choice. Each pure-C system library declares a callback table struct. The integration layer provides implementations that cast the `void *user_data` parameter to `game_ctx_t*` and dispatch to the appropriate module.

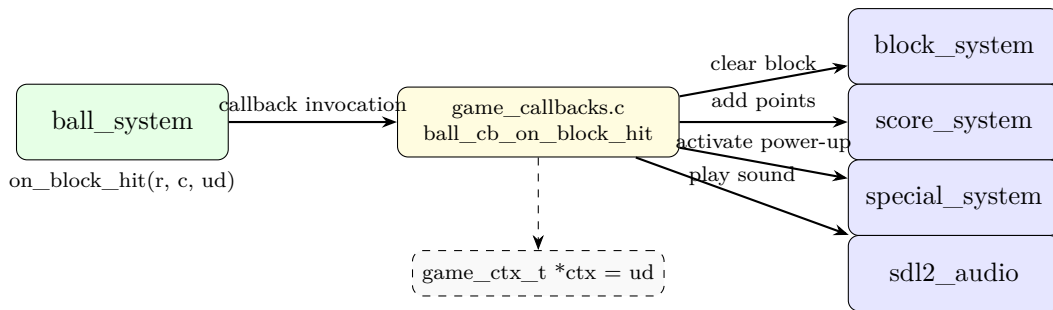


Figure 4: Callback injection flow. The ball system invokes `on_block_hit` via function pointer. The integration layer casts `user_data` to `game_ctx_t*` and dispatches to multiple target systems.

Example: ball block hit callback (from `game_callbacks.c`):

```

1 static int ball_cb_on_block_hit(int row, int col, int ball_index, void *ud)
2 {
3     game_ctx_t *ctx = ud;
4     int block_type = block_system_get_type(ctx->block, row, col);
5     int points = score_block_hit_points(block_type, row);
6     if (points > 0) {
7         score_system_env_t senv = {
8             .x2_active = special_system_is_active(ctx->special, SPECIAL_X2_BONUS),
9             .x4_active = special_system_is_active(ctx->special, SPECIAL_X4_BONUS),
10        };
11        score_system_add(ctx->score, (unsigned long)points, &senv);
12    }
13    switch (block_type) {
14        case DEATH_BLK: block_system_clear(ctx->block, row, col); return 1;
15        case BOMB_BLK: /* clear neighbors */ ...
16        case MULTIBALL_BLK: ball_system_split(ctx->ball, &senv); ...
17        /* 15+ more block type handlers */
18    }
19 }

```

This pattern eliminates the 50+ `extern` globals that coupled legacy modules. Each system library is independently testable by providing stub callbacks.

6.4 Mode Handlers (`game_modes.c`)

Registers `on_enter/on_update/on_exit` handlers for each of the 16 game modes with the state machine. Mode-specific logic includes:

GAME Core gameplay: updates ball, paddle, gun, eyedude, sfx, bonus spawning, timer count-down.

PAUSE Pauses the game loop, displays pause overlay.

BONUS Drives the 10-state bonus tally sequence, transitions to next level on completion.

PRESENTS/INTRO/DEMO/KEYS Attract mode screens with frame-driven state machines.

EDIT Level editor with palette, grid editing, play-test transitions.

DIALOGUE Modal text input (name entry, level selection, “words of wisdom”).

6.5 Rendering (`game_render.c` + `game_render_ui.c`)

`game_render.c` draws gameplay elements (blocks, balls, paddle, bullets, score, lives, specials, messages, timer, border, EyeDude). `game_render_ui.c` draws UI screens (presents, intro, demo, keys, bonus, highscore, editor, dialogue).

Each render function queries its corresponding system for render info and draws textures from the sprite catalog. The render dispatch is called once per frame with an interpolation alpha for smooth animation.

6.6 Input Dispatch (`game_input.c`)

Routes input actions to the appropriate handler based on the current game mode. Gameplay mode handles paddle movement, shooting, tilt, save/load. Attract modes handle start/cycle/quit. Editor mode handles draw/erase/palette/transforms.

6.7 Game Rules (`game_rules.c`)

Per-frame game logic: level completion detection (all breakable blocks cleared), ball death handling (decrement lives, reset ball or game over), bonus block spawning (random timing, 27 block types with probability distribution), and timer countdown (1-second decrement via frame accumulator).

7 Persistence

Four persistence modules handle file I/O with no SDL2 dependency.

7.1 High Score I/O (`highscore_io`)

JSON format replacing legacy binary `highScoreHeader` + `highScoreEntry` with `htonl` byte ordering. Hand-written JSON parser and emitter (no external dependency). Atomic writes via write-to-temp + rename. Two tables: global (shared) and personal (per-user).

7.2 Save Game I/O (`savegame_io`)

JSON format replacing legacy binary `saveGameStruct`. Saves level number, score, lives, specials state, block grid snapshot. Atomic writes.

7.3 Config I/O (`config_io`)

Minimal TOML format (flat key=value, no tables). Stores user preferences: speed, volume, nickname, sound/SFX enable, control mode. Located at `$XDG_DATA_HOME/xboing/config.toml`. CLI flags override config file values.

7.4 Path Resolution (`paths`)

XDG Base Directory path resolution (ADR-002). Centralizes all file path logic (12+ scattered callsites in legacy).

Read-only assets (levels, sounds) resolution priority:

1. Legacy env var (`XBOING_LEVELS_DIR`)
2. `XDG_DATA_DIRS` search
3. `XDG_DATA_HOME`
4. CWD fallback (`./levels/`)

Writable state (scores, saves) resolution priority:

1. Legacy env var (`XBOING_SCORE_FILE`)
2. Existing legacy file (`$HOME/.xboing-scores`)
3. XDG default (`$XDG_DATA_HOME/xboing/scores.dat`)

Design constraint: caller-provided buffers (no malloc, no static buffers). `paths_init_explicit()` accepts injected env values for deterministic testing.

8 Design Patterns

8.1 Opaque Context Pattern

Every system module defines a struct visible only in the `.c` file and exposes an opaque typedef in the `.h` file via forward declaration. The integration layer holds pointers to these opaque structs in `game_ctx_t`.

Consequences:

- Zero global mutable state across the entire modern codebase.
- Each module is independently testable via CMocka—create a context, inject stub callbacks, exercise API, destroy context.
- Header inclusion cycles are impossible: `game_context.h` uses only forward declarations.
- Multiple game instances could coexist (relevant for future replay comparison or headless testing).

8.2 Callback Injection

System libraries never call other system libraries directly. Side effects are communicated through function pointer tables (fig. 4). The integration layer provides implementations that bridge systems. This replaces the 50+ `extern` globals that coupled legacy modules.

Callback table sizes range from 1 (level system: `on_add_block`) to 8 (gun system: `check_block_hit`, `on_block_hit`, `check_ball_hit`, `on_ball_hit`, `check_eyedude_hit`, `on_eyedude_hit`, `on_sound`, `is_ball_waiting`).

8.3 Caller-Provided Buffers

Path resolution and render info functions write into caller-provided buffers rather than returning heap-allocated strings or using static buffers. This eliminates memory management errors and makes thread safety trivial.

8.4 Best-Effort Loading

Texture cache, audio cache, and cursor creation tolerate individual load failures. Missing assets are logged but do not abort initialization. Only structural failures (NULL renderer, `Mix_OpenAudio` failure) cause `create()` to return NULL.

8.5 Enum-Indexed Lookups

Colors, fonts, cursors, regions, and game modes use enum-indexed arrays for $O(1)$ lookup with compile-time type safety. This replaces string-based lookups for fixed-cardinality sets.

8.6 Environment Struct Pattern

Per-frame state from other systems is packaged into const structs (e.g., `ball_system_env_t`, `gun_system_env_t`, `score_system_env_t`) and passed to update functions. The system never caches these values. This makes data flow explicit and enables deterministic replay.

9 Testing Infrastructure

9.1 Test Framework

CMocka, discovered via `pkg-config`.

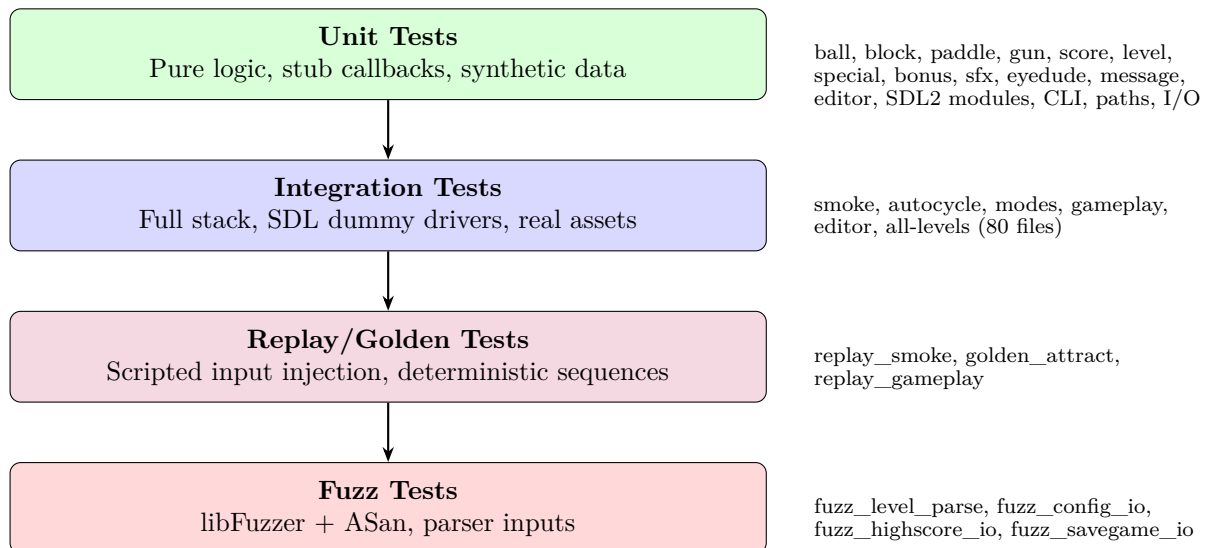


Figure 5: Test infrastructure layers. Each layer builds on the one below. Unit tests cover individual modules; integration tests exercise the full stack.

9.2 Headless Testing

SDL2 dummy drivers enable CI testing without display or audio hardware:

```
SDL_VIDEODRIVER=dummy # No window created, textures still allocate
SDL_AUDIODRIVER=dummy # Mix_OpenAudio succeeds silently
```

9.3 Sanitizer Builds

The `asan` preset builds all targets with `-fsanitize=address,undefined -fno-omit-frame-pointer`. Both debug and sanitizer configurations run in CI on every push and PR.

9.4 Fuzz Testing

Four libFuzzer harnesses target parser inputs (Clang only):

fuzz_level_parse Level file parser (character mapping, title/time parsing)

fuzz_config_io TOML config parser

fuzz_highscore_io JSON high score parser

fuzz_savegame_io JSON save game parser

Not registered as ctest targets (they run indefinitely). Manually invoked: `./build/tests/fuzz_level_parse -max_total_time=30`.

9.5 Integration Test Categories

Table 12: Integration test suite

Test	Scope
<code>test_integration_smoke</code>	Full <code>game_create/game_destroy</code> lifecycle
<code>test_integration_autocycle</code>	Attract sequence: PRESENTS→INTRO→...→HIGHSCORE→INTRO
<code>test_integration_modes</code>	Force-transition to each of 12 modes, tick N frames, verify no crashes
<code>test_integration_gameplay</code>	Enter GAME mode, verify state, block loading, ball/paddle, extended ticking
<code>test_integration_editor</code>	Enter EDIT mode, draw blocks, clear, play-test transitions
<code>test_replay_smoke</code>	Synthetic key injection, scripted mode transitions
<code>test_golden_attract</code>	Deterministic attract cycle mode transitions and timing
<code>test_replay_gameplay</code>	Scripted gameplay: start, paddle, shoot, pause, extended play
<code>test_integration_all_levels</code>	Load all 80 levels, verify parse, tick 100 frames each (120s timeout)

10 CI/CD

Five GitHub Actions workflows. `test.yml` and `lint.yml` run on every push to `master` and every PR; `docs.yml` is path-filtered to Markdown files (`**/*.md`). Two further workflows, `release.yml` (Debian package builds) and `biff-notify.yml` (team notifications), are not build or lint gates.

The per-workflow job and check breakdown is maintained in `docs/CI.md`; it is not duplicated here to avoid drift.

11 Module Dependency Map

12 Legacy Comparison

The modern codebase is larger in source LOC than legacy, reflecting the modular structure (function signatures, opaque context boilerplate, callback table definitions) and comprehensive error handling. The test suite exceeds the source in LOC.

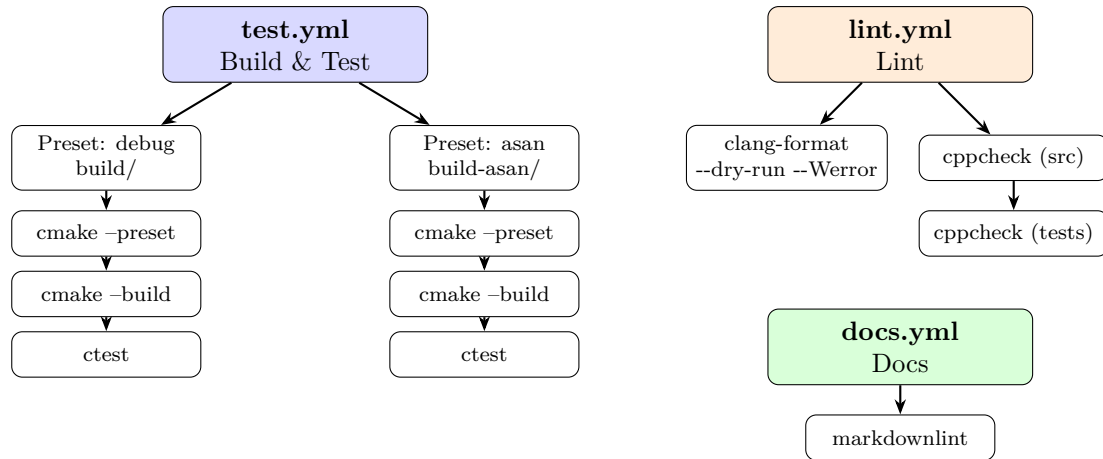


Figure 6: CI pipeline. Build matrix runs Debug and ASan configurations in parallel. Lint checks clang-format and cppcheck. Docs checks markdown.

Table 13: Structural comparison with legacy codebase

Aspect	Legacy	Modern
Global mutable variables	50+	0
X11/platform API calls in game logic	~800	0 (in system libs)
Audio drivers	12 (platform-specific)	1 (SDL2_mixer)
Build system	Makefile	CMake with presets
Static library decomposition	none	per-module
Test suite	none	unit, integration, replay, fuzz
CI	none	build + lint + docs

13 Architectural Decision Records Summary

All ADRs are recorded in `docs/DESIGN.md`, the single source of truth. Table 14 lists each by number and title; it is generated from those headers by `scripts/gen_adr_table.sh` and must not be edited by hand. See `docs/DESIGN.md` for the full rationale, status, and consequences of each decision.

Table 14: Architectural Decision Records (generated from `docs/DESIGN.md`)

ADR	Title
001	Use CMake as the sole build system
002	XDG Base Directory path resolution
003	FHS-compliant CMake install targets
004	SDL2 window and renderer architecture
005	SDL2 texture loading and caching
006	SDL2 TTF font rendering
007	SDL2 RGBA color system
008	SDL2 cursor management
009	SDL2 logical render regions
010	SDL2_mixer audio system
011	SDL2 input action mapping
012	Game state machine with function pointer dispatch

ADR	Title
013	Fixed-timestep game loop with accumulator
014	Command-line option parsing
015	Callback-based ball physics system
016	Pure C block grid system with diagonal cross-product collision
017	Pure C paddle system
018	Pure C gun/bullet system
019	Pure C score display system
020	Pure C level file loading system
021	Pure C special/power-up system
022	Pure C bonus tally sequence state machine
023	Pure C visual SFX state machine
024	Pure C EyeDude animated character system
025	Pure C presents/splash screen sequencer
026	Pure C intro and instructions screen sequencer
027	Pure C demo and preview screen sequencer
028	Pure C keys and editor controls screen sequencer
029	Pure C modal input dialogue system
030	Pure C high score display sequencer
031	Pure C message display system
032	JSON-based high score file I/O
033	JSON-based save game I/O
034	TOML-based user preferences file
035	cppcheck pass across entire codebase
036	clang-tidy pass with .clang-tidy configuration
037	Asset directory resolution for installed binaries
038	Save/load v2 — mid-flight resume deviation from original
039	Bonus screen visual fidelity — trust the goldens, not the source
040	Bonus coin count synced once at mode entry, not per pickup
041	Multi-user global high scores via setgid-games + flock
042	<code>game_active</code> is the score-submission gate, not a death sentinel
043	Bonus screen renderer reads from captured env, not live <code>ctx</code>
044	Failed level load ends the game via <code>HIGHSCORE</code> , not <code>ShutDown</code>
045	Warp speed velocity table — deviation from 1996 formula
046	macOS shared high-score path via <code>/Users/Shared</code>
047	Machine leaderboard is Debian-only; macOS shared board dropped
048	Leaderboard provider interface (proposed)
049	Sound on by default (deviation from the original)
050	Bonus interstitial predicts placement on the board the player competes on
051	Faithful editor cursors from the X11 cursor font
052	Every mode sets its cursor; plain arrow on menu/attract screens
053	Attract-screen <code>space</code> is once-per-frame; title <code>space</code> opens instructions
054	Attract high-score screen falls back to the personal board when the global board is empty
055	<code>game_active</code> is cleared on entering the title, closing the game-over leak into attract
056	A failed level load refuses <code>GAME</code> and returns to the title
057	Editor widens the logical canvas and window in lockstep
058	Editor erase reachable without a middle mouse button
059	Editor async dialogue mechanism
060	In-list palette highlight removed

ADR	Title
061	Digit-key and arrow-key palette selectors
062	Editor file-error handling shows a message and continues, instead of exiting
063	Ammo belt draws the raw count so it can't desync from the bonus
064	The level title is single-source across load, edit, and save
065	Editor play-test detection uses a dedicated flag, not previous-mode inference
066	Editor play-test board round-trip uses in-memory savegame capture/restore
067	DeadBall fidelity, play-test Q, and score-display leak
068	Asset resolution consults the compiled install prefix
069	Mid-play special-block spawner is throttled like the original
070	Roamer and drop blocks move; direction mapping drops a 1996 bug
071	Gameplay frame→tick mapping is 1:1; the ~22.5 s ball auto-launch is faithful, not a bug
072	-debug is now functional; the = skip-level cheat is restored
073	Using the = skip-level cheat disqualifies the session from ALL high-score boards
074	Explosion timer uses >=; the skip-level sweep moves into <code>block_system</code>

14 Glossary

Legacy The original X11/Xlib codebase (1993–1996, `original/` directory).

Modern The SDL2-based codebase (`src/`, `include/`).

Opaque context Heap-allocated struct hidden behind a forward-declared typedef; accessed only via public API functions.

Callback injection Providing function pointer tables at module creation time so the module can communicate side effects without direct dependencies on other modules.

Environment struct A const struct carrying per-frame state from other systems, passed to update functions. Replaces extern globals.

FNV-1a Fowler–Noll–Vo hash function variant 1a; used for texture and audio cache hash maps.

XDG X Desktop Group Base Directory Specification; standard Linux paths for user data, config, and cache.

FHS Filesystem Hierarchy Standard; standard Linux install paths (`/usr/share/`, `/usr/bin/`).

Sequencer A UI module that advances through a state machine per frame, exposing render data through query functions. Does not render directly.

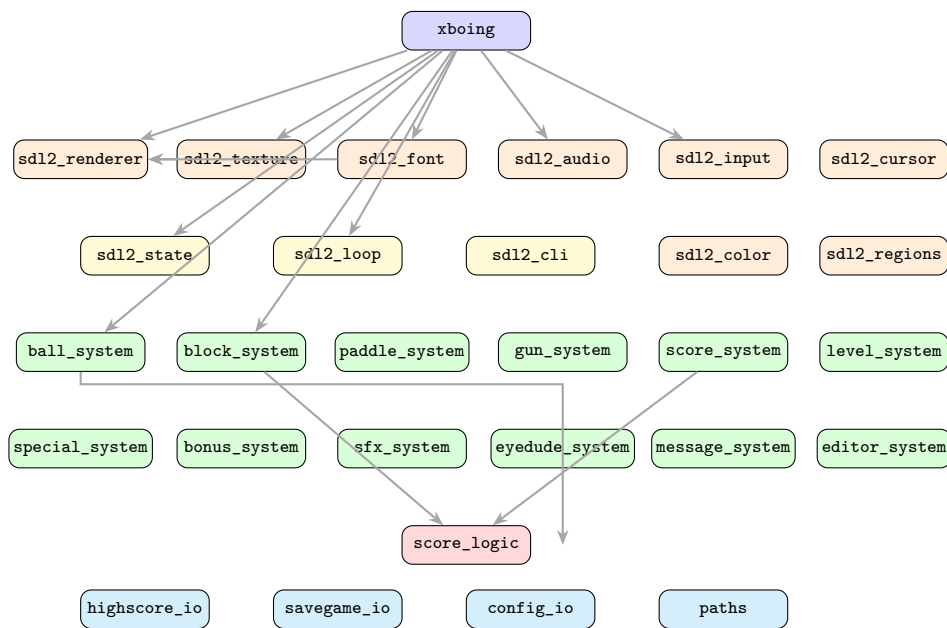


Figure 7: Module dependency graph. The executable links all libraries. `score_logic` is the only shared dependency between game systems (`block_system` for hit points, `score_system` for multiplier/threshold). `sd12_texture` and `sd12_font` depend on `sd12_renderer` for the `SDL_Renderer*` handle.